ziggy

Book the place. Skip the noise.

Brand & Build Blueprint

Identity, colour system, frontend design and backend architecture for the Ziggy apartment booking platform.

Version 1.0 | November 2026 | Frontend on Vercel | Backend and PostgreSQL on Render

SECTION 01

Brand foundation

Ziggy is an apartment booking platform for guests who want a clear, trustworthy way to find and reserve a place to stay, and for hosts who manage the listings. The brand should feel like a well-run property office: calm, precise and quietly confident.

Element	Decision
Name	**Ziggy** (set by you). Always written in lowercase in the logo, capitalised in running text.
Positioning	The calm, no-nonsense way to book an apartment.
Tagline (proposed)	Book the place. Skip the noise.
Audience	Guests booking short and mid-term stays. Hosts and managers listing apartments. Admin staff.
Personality	**Clear** - says what it means. **Trustworthy** - honest prices, no surprises. **Warm** - human, never salesy. **Efficient** - few steps, fast pages.

Design principles

Principle	What it means in practice
Flat and solid	No gradients anywhere: not in backgrounds, buttons, text or images. Surfaces are solid colour with thin borders. Shadows are used only on floating layers (menus, modals).
One accent	A single terracotta accent marks the main action on a screen. Everything else is Ink, Sand and Stone.
Photos carry the colour	The interface stays neutral so apartment photography is the most colourful thing on the page.
Space and clarity	Generous whitespace, a 4px grid, short labels, one primary action per view.
Accessible by default	WCAG AA contrast, visible focus states, full keyboard use, readable at 16px body size.

Voice and tone

Do	Do not
Your stay is confirmed. Check-in is Friday from 14:00.	Awesome!!! Get ready for an AMAZING trip!
Those dates are taken. Try 14 to 17 November.	Oops, something went wrong :(
Short sentences, plain words, real numbers.	Marketing superlatives, exclamation marks, emojis in the interface.

Before you launch: check the name and mark
I designed this logo from scratch, but I cannot verify that the name "Ziggy" or the mark is unused anywhere in the world. Several businesses use similar names. Before you invest in printing or ads, run a trademark search in your market and in the WIPO Global Brand Database, check domain and social handles, and run a reverse image search on the mark. A short consult with a trademark attorney is worth it.

SECTION 02

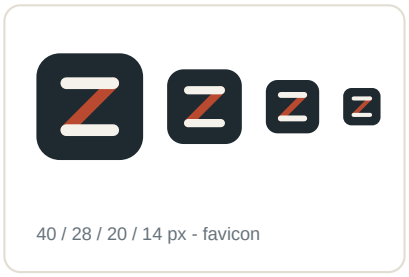
Logo

The Ziggy mark is a rounded tile holding a single "Z" drawn as one continuous zigzag. The diagonal stroke is the only accent-coloured element: it reads as a staircase between floors, a roofline, and the "zig" of the name. The wordmark is set in bold lowercase for a friendly, steady feel.

Primary lockup - on Sand

Reversed lockup - on Ink

On Accent (rare)



Mark only - app icon, favicon

Clear space

Usage rules

Do	Do not
Use the primary lockup on Sand or white, the reversed lockup on Ink.	Add gradients, shadows, outlines or glows.
Keep clear space of half the mark width on every side.	Stretch, rotate, or recolour the mark beyond the palette.
Minimum sizes: lockup 96px wide, mark alone 16px.	Place the lockup on busy photos without an Ink or Sand panel behind it.
Use the mark alone for favicon, app icon and social avatar.	Change the wordmark to uppercase or another typeface.

SECTION 03

Colour system

A restrained, warm-neutral palette with a single accent. Every pair below was checked for contrast.

	Name	HEX	RGB	Used for
	Ink	#1F2A30	31, 42, 48	Text, primary buttons, footer, dark panels
	Ink Soft	#2E3B42	46, 59, 66	Hover on Ink, dark sidebar active state
	Sand	#F5F2EB	245, 242, 235	Page background, hero and soft panels
	Stone	#E3DED3	227, 222, 211	Borders, dividers, image placeholders
	Terracotta	#B94A30	185, 74, 48	The single accent: main call to action, links, key highlights
	Terracotta Dark	#9A3D27	154, 61, 39	Hover and pressed state of the accent
	Slate	#66727A	102, 114, 122	Secondary text, captions, placeholders
	White	#FFFFFF	255, 255, 255	Cards, inputs, modals

Status colours

State	Text	Background tint	Where
Success	Green #3E7B5A	#DCEBE2	Confirmed bookings, saved messages
Warning	Amber #A8691A	#F3E5CC	Pending bookings, expiring holds
Error	Red #A63A2B	#F1D9D4	Cancelled bookings, form errors

Balance on a screen

Roughly how much of any screen each colour should cover. Accent is a highlighter, not a theme.

Contrast checks

Pair	Ratio	Result
Ink text on Sand	13.1 : 1	AA and AAA for all text
White text on Terracotta button	5.1 : 1	AA for all text
Terracotta text/link on Sand	4.6 : 1	AA for body text, use bold for small links
Slate text on Sand	4.4 : 1	AA for body text

Rule for developers
No `linear-gradient`, `radial-gradient` or gradient text anywhere in the codebase. If a design needs depth, use a 1px Stone border or a flat tint instead.

SECTION 04

Typography and components

Headings use Manrope (geometric, friendly) and body text uses Inter (very readable at small sizes). Both are free on Google Fonts. This PDF uses Helvetica as a stand-in because it cannot embed web fonts.

Style	Font	Size / line height	Weight	Use
Display	Manrope	48 / 52	700	Home hero headline
H1	Manrope	32 / 38	700	Page titles
H2	Manrope	24 / 30	700	Section titles
H3	Manrope	18 / 26	600	Card titles, panel headings
Body	Inter	16 / 26	400	Paragraphs, descriptions
Small	Inter	14 / 22	400 / 500	Labels, helper text, table cells
Caption	Inter	12 / 18	500	Badges, metadata

Core components

Reserve stay

Primary

Book now

Accent - one per screen

Sign in

Secondary

Check-in

Select a date

Focus

12 Nov 2026

Input - 1px Stone border, Ink border on focus

Confirmed Pending Cancelled

Status badges - tinted background, solid text

photo

Sunlit 2-bed, Downtown

Central district | 4 guests

$120 / night

View

Radius 8px (controls) / 12px (cards) | 4px spacing grid | 1px Stone borders | no gradients, no gl

SECTION 05

Frontend plan

A React single-page app built with Vite, styled with Tailwind using the Ziggy tokens, and deployed to Vercel. It talks to the API through one Axios instance configured with `VITE_API_URL`.

Pages and routes

Route	Page	Access	Contents
/	Home	Public	Hero with search (location, dates, guests), featured apartments, how it works, footer
/apartments	Listings	Public	Filter bar and sidebar (price, bedrooms, amenities), result grid, pagination, empty and loading states
/apartments/:id	Apartment details	Public	Photo gallery, description, amenities, location, booking card with date range and guests, price breakdown
/login, /register	Auth	Public	Forms with inline validation; redirect back to the page the visitor came from
/bookings	My bookings	Signed in	Upcoming and past stays, status badges, cancel action
/admin	Dashboard	Admin	Stat cards, bookings table with status filter, apartments table
/admin/apartments/new, /:id/edit	Apartment form	Admin	Create and edit listing: details, price, amenities, image URLs, availability
*	Not found	Public	Short message and a link home

Folder structure

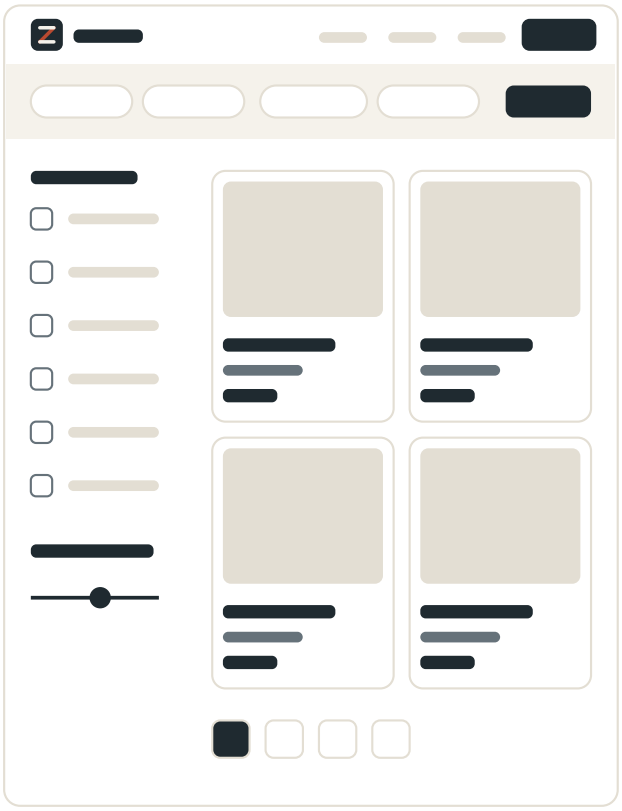
```
client/
  index.html   vercel.json   tailwind.config.js   .env   .env.example
  src/
    main.jsx   App.jsx   routes.jsx
    api/          axios.js  auth.js  apartments.js  bookings.js
    context/      AuthContext.jsx
    hooks/        useApartments.js  useBookings.js
    components/
      layout/     Navbar.jsx  Footer.jsx  Container.jsx  ProtectedRoute.jsx  AdminRoute.jsx
      ui/         Button.jsx  Input.jsx  Badge.jsx  Modal.jsx  Toast.jsx  Skeleton.jsx  EmptyState.jsx
      apartments/ ApartmentCard.jsx  FilterBar.jsx  Gallery.jsx  AmenityList.jsx
      booking/    BookingCard.jsx  DateRangePicker.jsx  PriceBreakdown.jsx
    pages/        Home  Apartments  ApartmentDetails  Login  Register  MyBookings  NotFound
      admin/      Dashboard.jsx  ApartmentForm.jsx
    styles/       index.css
```

SECTION 06

Wireframes: home and listings

Low-fidelity layouts that show structure and the use of colour. Real photography replaces the grey blocks.

Home



Listings with filters

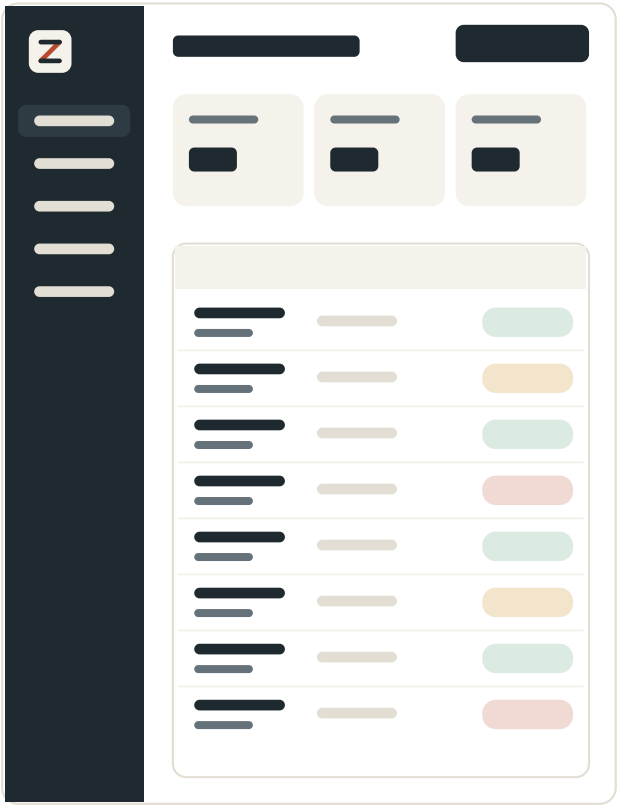
Home	Listings
Hero on Sand with a single search bar (where, dates, guests) and one Terracotta search button. Featured stays below as three cards. Ink footer. No decorative shapes.	**Filter chips** across the top, a checkbox and price sidebar on the left, a two-column card grid on the right. Sidebar becomes a slide-up sheet on mobile. Pagination uses Ink for the active page.

SECTION 07

Wireframes: details and admin

The two screens that carry the most business value: where a guest decides, and where staff keep things running.

Apartment details



Admin dashboard

Apartment details	Admin dashboard
Gallery with one large and two small photos. Description and amenities on the left. **Booking card** on the right stays sticky on desktop: dates, guests, price breakdown and the only Terracotta button on the page. On mobile the card collapses to a bottom bar with the price and a Reserve button.	**Ink sidebar** for navigation, three stat cards (bookings, revenue, occupancy), then a bookings table with tinted status badges. Filters and search above the table. Apartment management lives under the same shell.

SECTION 08

Frontend behaviour and theme setup

Key behaviours

- **Auth:** JWT kept in localStorage, attached by an Axios request interceptor. A response interceptor logs the user out on a 401. `ProtectedRoute` and `AdminRoute` guard pages.
- **Booking flow:** pick dates and guests, see the price breakdown update live, press Reserve. If not signed in, go to login and return to the same apartment with dates preserved.
- **Date rules:** past dates disabled, check-out after check-in, guests capped at the apartment maximum. The server is still the source of truth.
- **Feedback:** skeleton loaders for lists, toast messages for success and errors, inline errors under fields, friendly message when dates are taken.
- **Performance:** route-level code splitting, lazy-loaded images with fixed aspect ratio, debounced search input.
- **Accessibility:** semantic landmarks, labelled inputs, focus ring in Ink with 2px offset, keyboard-friendly date picker and gallery, alt text on every photo.
- **Responsive:** mobile first. Breakpoints at 640, 768 and 1024px. Cards go from one to two to three columns.

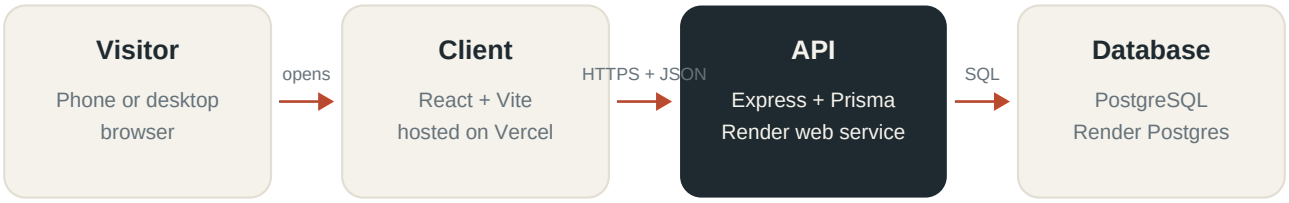
Tailwind theme (colours only)

```
// client/tailwind.config.js
export default {
  content: ['./index.html', './src/**/*.{js,jsx}'],
  theme: {
    extend: {
      colors: {
        ink:    { DEFAULT: '#1F2A30', soft: '#2E3B42' },
        accent: { DEFAULT: '#B94A30', dark: '#9A3D27' },
        sand:   { DEFAULT: '#F5F2EB', deep: '#E3DED3' },
        slate:  '#66727A',
        success: '#3E7B5A', warning: '#A8691A', danger: '#A63A2B',
      },
      borderRadius: { control: '8px', card: '12px' },
      fontFamily: { heading: ['Manrope', 'sans-serif'], body: ['Inter', 'sans-serif'] },
      boxShadow: { overlay: '0 8px 24px rgba(31,42,48,0.12)' },  // menus and modals only
    },
  },
}
```

SECTION 09

Backend plan

A REST API built with Node.js and Express, using Prisma as the ORM over PostgreSQL. It runs as a web service on Render with a managed Render Postgres database.



The client never talks to the database. Only the API holds the database URL and the JWT secret.

Folder structure

```
server/
  .env   .env.example   package.json
  prisma/
    schema.prisma   seed.js   migrations/
  src/
    index.js                starts the server on process.env.PORT
    app.js                  express app, middleware, routes
    config/env.js           loads and validates environment variables
    lib/prisma.js           single shared PrismaClient
    middleware/             auth.js  role.js  validate.js  errorHandler.js
    routes/                 auth.routes.js  apartments.routes.js  bookings.routes.js  health.routes.js
    controllers/            auth.js  apartments.js  bookings.js
    services/               booking.service.js   (overlap check inside a transaction)
    utils/                  jwt.js  pricing.js
```

Data model

Model	Fields	Rules
User	id (uuid), name, email, password, role (USER | ADMIN), createdAt, updatedAt	Email unique. Password stored as a bcrypt hash and never returned.
Apartment	id, title, description, location, pricePerNight, bedrooms, bathrooms, maxGuests, amenities[], images[], isAvailable, createdAt, updatedAt	Indexed on location and pricePerNight. Only admins create, edit or delete.
Booking	id, userId, apartmentId, checkIn, checkOut, guests, totalPrice, status (PENDING | CONFIRMED | CANCELLED), createdAt, updatedAt	Indexed on apartmentId + dates. Cancelled bookings free the dates.

SECTION 10

API reference

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Create an account, returns user and JWT
POST	/api/auth/login	Public	Sign in, returns user and JWT
GET	/api/auth/me	Signed in	Current user from the token
GET	/api/apartments	Public	List with search and filters: location, minPrice, maxPrice, guests, checkIn, checkOut, page, limit
GET	/api/apartments/:id	Public	One apartment
POST	/api/apartments	Admin	Create apartment
PUT	/api/apartments/:id	Admin	Update apartment
DELETE	/api/apartments/:id	Admin	Delete apartment
POST	/api/bookings	Signed in	Create booking: validates dates and guests, blocks overlaps, calculates total
GET	/api/bookings/my	Signed in	The current user's bookings
PATCH	/api/bookings/:id/cancel	Owner / Admin	Cancel a booking
GET	/api/bookings	Admin	All bookings with filters
GET	/api/health	Public	Health check including a quick database query (used by Render)

Booking logic (no double bookings)

- Validate input: check-in is today or later, check-out is after check-in, guests is at most the apartment maximum, apartment exists and is available.
- Open a Prisma transaction. Look for any non-cancelled booking for the same apartment where `existing.checkIn < new.checkOut` and `existing.checkOut > new.checkIn`.
- If one exists, stop and return 409 with a clear message. Otherwise compute `nights x pricePerNight` on the server and create the booking in the same transaction.
- Never trust a price sent by the client. Always compute it on the server.

Security and reliability

- Passwords hashed with bcrypt (cost 12). JWT signed with `JWT_SECRET`, expiry from `JWT_EXPIRES_IN`, sent as a Bearer token.
- Helmet headers, CORS limited to `CLIENT_URL`, rate limit on all routes with a stricter limit on login and register.
- express-validator on every write route. Prisma uses parameterised queries. A global error handler returns consistent JSON and hides stack traces in production.
- Secrets live only in the Render and Vercel dashboards. The `.env` files are git-ignored and only `.env.example` is committed.

SECTION 11

Environment, local development and deployment

Environment variables

App	Variable	Local value	Production value
Server	PORT	5000	Set automatically by Render
Server	DATABASE_URL	postgresql://postgres:password@localhost:5432/apartment_booking	Render Internal Database URL
Server	JWT_SECRET	any long random string	A different long random string
Server	JWT_EXPIRES_IN	7d	7d
Server	CLIENT_URL	http://localhost:5173	Your Vercel URL
Server	NODE_ENV	development	production
Client	VITE_API_URL	http://localhost:5000/api	Render URL + /api

One command to run everything

```
// package.json (repo root)
"scripts": {
  "dev": "concurrently -k -n client,server \"npm:dev:client\" \"npm:dev:server\"",
  "dev:client": "npm run dev --prefix client",
  "dev:server": "npm run dev --prefix server",
  "install:all": "npm install && npm install --prefix client && npm install --prefix server",
  "build": "npm run build --prefix client"
}
```

Works the same in Windows PowerShell, macOS and Linux. Client on http://localhost:5173, API on http://localhost:5000.

Deployment settings

Service	Setting	Value
Render Postgres	Name / database	apartment-booking-db / apartment_booking
Render web service	Root directory	server
	Build command	npm install && npx prisma migrate deploy
	Start command	npm start
	Health check path	/api/health
Vercel	Root directory	client
	Build / output	npm run build / dist
	Rewrites	vercel.json sends every path to index.html so React Router works on refresh

Deploy order: database, then API, then client. After the client has a URL, set `CLIENT_URL` on Render to that URL and redeploy the API. Render's free web service sleeps when idle, so the first request after a quiet period can take a while.

Build roadmap

Phase	Scope	Done when
1 Foundations	Monorepo, root dev script, env files, Tailwind tokens, UI kit components	`npm run dev` starts both apps
2 Backend	Prisma schema and migration, auth, apartments, bookings, health, seed script	All endpoints work with seeded data
3 Frontend core	Home, listings, details, login, register, auth context	A guest can browse and sign in
4 Booking and admin	Booking card, my bookings, admin dashboard and forms	End-to-end booking and cancel works
5 Deploy and polish	Render and Vercel, CORS, error states, accessibility pass	Live URLs work from a phone
Later	Online payments, reviews, image upload, email confirmations	After the core is stable

Next step
Once you approve this direction, I will turn it into a single master build prompt with the Ziggy colours, fonts, components, and the PostgreSQL backend baked in, so your coding tool generates the site on-brand from the first run.